

BNO055 IMU Sensor

Radxa Zero 3w Integration Guide (.VENV)

1 Overview

This guide explains how to interface a BNO055 9-axis IMU sensor with a Radxa Zero 3w using only Python standard library modules.

The BNO055 is a smart IMU that contains:

- 3-axis accelerometer (measures acceleration)
- 3-axis gyroscope (measures rotation rate)
- 3-axis magnetometer (measures magnetic field)
- Internal microcontroller that performs sensor fusion

Sensor fusion combines all 9 axes to produce absolute orientation data such as heading, roll, and pitch.

Communication is done over I2C using the Linux device file: `/dev/i2c-5`

2 How I2C Communication Works

I2C is a two-wire communication protocol:

- SDA = data line
- SCL = clock line

Each device on the bus has an address. The BNO055 default address is `0x28`.

In Linux, I2C devices are accessed as files (for example: `/dev/i2c-5`). We open this file and use `ioctl()` to tell the kernel which device address we want to talk to.

3 Device Information

Chip ID	0xA0 (verifies correct sensor)
I2C Address	0x28
I2C Bus	/dev/i2c-5
Python Version	3.11.2
Libraries Used	os, fcntl, struct, time

The Chip ID is stored in register 0x00. If this does not return 0xA0, communication is not working correctly.

4 Wiring

BNO055 Pin	Radxa Pin	Notes
VIN	3.3V	Do NOT use 5V
GND	GND	Common ground required
SDA	SDA (Bus 5)	I2C data line
SCL	SCL (Bus 5)	I2C clock line

Both devices must share a common ground.

5 Check Device on I2C

Run:

```
sudo i2cdetect -r -y 5
```

This scans bus 5 and lists detected I2C devices. You should see 28 in the grid, which corresponds to address 0x28.

If nothing appears:

- Check wiring
- Confirm 3.3V power
- Confirm I2C bus 5 is enabled

6 Verify Chip ID

This script confirms communication with the sensor.

```
import fcntl, os

I2C_SLAVE = 0x0703  # Linux constant for selecting I2C device

fd = os.open('/dev/i2c-5', os.O_RDWR)

# Tell kernel we want to talk to device 0x28
fcntl.ioctl(fd, I2C_SLAVE, 0x28)

# Write register address 0x00 (CHIP_ID register)
os.write(fd, bytes([0x00]))

# Read 1 byte from that register
chip_id = os.read(fd, 1)

print(hex(chip_id[0]))  # should print 0xa0
```

```
os.close(fd)
```

What this does:

- Opens the I2C device file
- Selects the BNO055 address
- Requests register 0x00
- Reads the returned value
- Prints the chip ID

If this prints 0xa0, the sensor is responding correctly.

7 Understanding the Full Sensor Script

The full script does three main things:

1. Sets operation mode to NDOF (sensor fusion mode)
2. Reads orientation (heading, roll, pitch)
3. Reads acceleration and temperature

7.1 Setting Operation Mode

Register 0x3D controls operation mode.

Value 0x0C sets NDOF mode:

- Uses accelerometer + gyroscope + magnetometer
- Outputs absolute orientation

The delay (0.8 seconds) allows the sensor to switch modes.

7.2 Reading Euler Angles

Registers 0x1A to 0x1F contain:

- Heading (Yaw)
- Roll
- Pitch

Each value:

- Is a signed 16-bit integer
- Must be divided by 16 to convert to degrees

7.3 Reading Accelerometer Data

Registers 0x08 to 0x0D contain:

- X acceleration
- Y acceleration
- Z acceleration

Each value:

- Is a signed 16-bit integer
- Must be divided by 100 to convert to m/s^2

7.4 Reading Temperature

Register 0x34 stores temperature in degrees Celsius as a signed byte.

8 Full Sensor Script

```
import fcntl, os, time, struct

I2C_SLAVE = 0x0703
fd = os.open('/dev/i2c-5', os.O_RDWR)
fcntl.ioctl(fd, I2C_SLAVE, 0x28)

def read_reg(reg, length=1):
    os.write(fd, bytes([reg]))
    return os.read(fd, length)

def write_reg(reg, value):
    os.write(fd, bytes([reg, value]))

write_reg(0x3D, 0x0C) # Set NDOF mode
time.sleep(0.8)

while True:

    data = read_reg(0x1A, 6)
    heading = struct.unpack('<h', data[0:2])[0] / 16.0
    roll    = struct.unpack('<h', data[2:4])[0] / 16.0
    pitch   = struct.unpack('<h', data[4:6])[0] / 16.0

    data = read_reg(0x08, 6)
    ax = struct.unpack('<h', data[0:2])[0] / 100.0
    ay = struct.unpack('<h', data[2:4])[0] / 100.0
    az = struct.unpack('<h', data[4:6])[0] / 100.0

    temp = struct.unpack('b', read_reg(0x34))[0]

    print("Heading:", heading)
```

```
print("Roll:", roll)
print("Pitch:", pitch)
print("Accel:", ax, ay, az)
print("Temp:", temp)
print("-" * 40)

time.sleep(0.5)

os.close(fd)
```

9 Calibration

Register 0x35 contains calibration status.

Each 2-bit value represents calibration level:

- 0 = not calibrated
- 1 = low
- 2 = medium
- 3 = fully calibrated

```
calib = read_reg(0x35)[0]
sys    = (calib >> 6) & 0x03
gyro   = (calib >> 4) & 0x03
accel  = (calib >> 2) & 0x03
mag     = (calib >> 0) & 0x03

print("Sys:", sys, "Gyro:", gyro, "Accel:", accel, "Mag:", mag)
```

To calibrate:

- Keep still (gyro)
- Rotate slowly on all sides (accelerometer)
- Move in figure-8 pattern (magnetometer)

10 Common Errors

- Bad file descriptor → Use `os.open(..., os.O_RDWR)`
- No such device → Check wiring and bus number
- Wrong chip ID → Try address 0x29
- Not detected in `i2cdetect` → Check 3.3V and GND